

1. Resumen Ejecutivo y Arquitectura General

El proyecto consiste en una aplicación web (SaaS) diseñada para el aprendizaje avanzado del idioma coreano mediante un Sistema de Repetición Espaciada (SRS). Su principal innovación radica en el uso de Inteligencia Artificial como analizador morfológico automatizado, su sistema de doble clasificación (gramatical y semántica) y una arquitectura de base de datos dividida en un "Diccionario Global" y "Colecciones de Usuario" para maximizar la eficiencia y reducir costes de API.

Stack Tecnológico Recomendado

- **Backend:** PHP con Laravel (ideal para su ORM Eloquent, manejo de relaciones complejas, RBAC y sistema de colas).
- **Frontend:** React, Vue.js o Blade (Laravel) con diseño responsivo (PWA).
- **Base de Datos:** PostgreSQL o MySQL.
- **Integración IA:** API de OpenAI (GPT-4o) o Anthropic (Claude 3.5 Sonnet) configurada para devolver respuestas en formato JSON estricto.

2. Modelo de Datos Relacional

El núcleo del sistema es la separación entre el conocimiento universal (Diccionario) y el progreso individual (Usuarios).

A. Diccionario Global (Conocimiento Base)

Almacena los elementos morfológicos del idioma sin duplicados.

Tabla	Descripción	Columnas Principales
entities	Unidades mínimas de significado (raíces, partículas, palabras simples).	id, text, type (root, particle, word), meaning, status (pending_review, verified).
compounds	Palabras aglutinadas formadas por varias entidades (ej. verbo + partícula).	id, full_text, translation, status.

<code>compound_entity</code>	Tabla puente que relaciona el compuesto con las entidades que lo forman.	<code>compound_id</code> , <code>entity_id</code> , <code>position_order</code> .
<code>tags</code>	Etiquetas semánticas situacionales.	<code>id</code> , <code>name</code> (ej. Restaurante, Saludos).
<code>taggables</code>	Tabla polimórfica para asignar etiquetas a <code>entities</code> o <code>compounds</code> .	<code>tag_id</code> , <code>taggable_id</code> , <code>taggable_type</code> .

B. Progreso de Usuario (Sistema SRS)

Registra las interacciones de los usuarios con el Diccionario Global.

Tabla	Descripción	Columnas Principales
<code>users</code>	Sistema de cuentas y autenticación.	<code>id</code> , <code>name</code> , <code>email</code> , <code>role</code> (user, admin).
<code>user_progress</code>	Tabla pivot entre el usuario y un elemento de estudio (entidad o compuesto).	<code>user_id</code> , <code>item_id</code> , <code>item_type</code> , <code>next_review_date</code> , <code>ease_factor</code> , <code>interval</code> .
<code>study_logs</code>	Registro histórico inmutable de cada intento (tarjeta repasada).	<code>id</code> , <code>user_id</code> , <code>item_id</code> , <code>is_correct</code> , <code>time_taken_ms</code> , <code>created_at</code> .

3. Pipeline de Inteligencia Artificial (NLP) e Ingesta

Cuando un usuario introduce un término nuevo, el sistema ejecuta un flujo optimizado para ahorrar costes y garantizar la precisión de los datos.

Lógica de Caché (Ahorro de API)

1. El usuario introduce un término (ej. 먹었어요).
2. El backend consulta la tabla **compounds** y **entities**.
3. **Hit (Existe)**: Se crea el registro en **user_progress** para ese usuario. No se llama a la IA. Coste: \$0.
4. **Miss (No existe)**: Se invoca a la API del modelo de lenguaje.

Analizador Morfológico (Prompt Output)

La IA recibe instrucciones estrictas para diseccionar la lengua aglutinante y devuelve una estructura de datos lista para ser parseada por el backend.

```
{
  "full_compound": {
    "text": "먹었어요",
    "translation": "Comí (Formal estándar)",
    "tags": ["Comida", "Pasado", "Formal"]
  },
  "components": [
    {
      "text": "먹다",
      "type": "verb_root",
      "meaning": "Comer",
      "tags": ["Comida"]
    },
    {
      "text": "었어요",
      "type": "conjugation_particle",
      "meaning": "Marcador de pasado formal estándar",
      "tags": ["Gramática", "Pasado"]
    }
  ]
}
```

El backend procesa este JSON:

1. Inserta "먹다" y "었어요" en **entities** con estado **pending_review**.
2. Inserta "먹었어요" en **compounds** con estado **pending_review**.
3. Crea las relaciones en **compound_entity**.
4. Asigna las etiquetas correspondientes.

4. Backoffice y Control de Calidad (Data Governance)

Para evitar la degradación de la base de datos por alucinaciones de la IA, se implementa un sistema de Control de Acceso Basado en Roles (RBAC).

- **Rol Usuario (user):** Puede añadir vocabulario (que dispara a la IA si es nuevo), repasar sus tarjetas y ver sus estadísticas. Ve todos sus términos, independientemente del estado de revisión global.
- **Rol Administrador (admin):** Tiene acceso a un panel de control exclusivo (Dashboard de Gestión).
 - **Cola de Revisión:** Vista de todos los registros en `entities` y `compounds` donde `status = 'pending_review'`.
 - **Herramientas de Curación:** Interfaz para corregir traducciones, reasignar etiquetas, o modificar la disección morfológica si la IA se equivocó separando la raíz de la partícula.
 - **Aprobación:** Al marcar como `verified`, el término se considera canon en el Diccionario Global.

5. Motor SRS y Analítica de Datos

El sistema no solo programa repasos (basado en algoritmos probados como SuperMemo 2 o FSRs), sino que procesa la tabla `study_logs` para generar inteligencia educativa.

Métricas Calculadas

- **Tasa de Acierto Individual:** $(\text{Aciertos de la entidad X} / \text{Total de intentos de la entidad X}) * 100$.
- **Rendimiento Semántico:** Porcentaje de acierto agrupando todas las entidades que comparten el tag "Educación".
- **Rendimiento Estructural:** Porcentaje de acierto agrupando por el campo `type` (ej. "Partículas de Tema" vs "Raíces Verbales").
- **Análisis Cruzado:** Capacidad de consultar intersecciones. Ejemplo: *"Rendimiento en verbos (Estructural) dentro del contexto de Tiendas (Semántico)"*.

Al tener las palabras aglutinadas separadas en la base de datos, el sistema de estadísticas puede identificar si el usuario está fallando de manera consistente en la memorización de la partícula de lugar "에", independientemente del sustantivo al que vaya unida.

6. Requisitos No Funcionales (RNF)

- **Concurrencia y Rendimiento:** La generación del siguiente lote de tarjetas de repaso mediante consultas SQL (búsqueda en `user_progress` donde `next_review_date <= NOW()`) debe resolverse en menos de 200 milisegundos.

- **Seguridad:** Las rutas del panel de administración (`/api/admin/*`) estarán protegidas por *middleware* de verificación de roles. Los tokens de autenticación se gestionarán mediante JWT o Laravel Sanctum.
- **Integridad Referencial:** Eliminación en cascada (*Cascade Delete*) controlada. Si un administrador elimina una entidad falsa generada por error, debe eliminarse de las colecciones de los usuarios que la tuvieran guardada.